

Exception Aufgaben

Concat Strings

Schreibe eine statische Methode, die zwei Strings als Parameter akzeptiert und diese konkateniert* zurückgibt.

Implementiere eine Null-Prüfung für beide Eingabestrings und werfe eine geeignete Exception, falls einer oder beide Strings null sind.

```
public static String concat(String str1, String str2)
```

Schreibe einen Unit Test mit 100% Abdeckung für die Methode.

* In Java bezieht sich "konkateniert" auf die Verkettung von Zeichenfolgen oder anderen Datenstrukturen.

Beispiel: "Elon" + "Musk"; // Konkatenation zweier Strings

Das Konkatenieren ist ein Prozess, bei dem zwei oder mehr Zeichenfolgen oder Datenstrukturen zusammengefügt werden, um eine einzige Zeichenfolge oder Datenstruktur zu erstellen.

Concat Strings: Lösung

```
class StringConcatenator {  
  
    static String concat(String str1, String str2) {  
        if (str1 == null || str2 == null) {  
            throw new NullPointerException("Parameter cannot be null");  
        }  
        return str1 + str2;  
    }  
}
```

```
class StringConcatenatorTest {  
  
    @Test  
    void testConcatWithNonNullStrings() {  
        String result = StringConcatenator.concat("Hello", "World");  
  
        assertEquals("HelloWorld", result);  
    }  
  
    @Test  
    void testConcatWithFirstStringNull() {  
        assertThrows(NullPointerException.class, () -> StringConcatenator.concat(null, "World"));  
    }  
  
    @Test  
    void testConcatWithSecondStringNull() {  
        assertThrows(NullPointerException.class, () -> StringConcatenator.concat("Hello", null));  
    }  
  
    @Test  
    void testConcatWithBothStringsNull() {  
        assertThrows(NullPointerException.class, () -> StringConcatenator.concat(null, null));  
    }  
}
```

Double Converter

Erstelle eine eigene **Checked** Exception mit dem Namen IsNotANumberException.

Schreibe eine Methode, die ein String als Parameter akzeptiert.

`static double toDouble(String number) throws IsNotANumberException`

Die Methode prüft, ob der übergebene String als double umgewandelt werden kann. Falls nicht wirft sie eine IsNotANumberException, ansonsten wird der String als double zurückgegeben (eventuell muss du hierzu eine Java-Exception abfangen und umwandeln).

Rufe die Methode in einer main-Methode auf: einmal für den Exception-Fall und einmal ohne Exception-Fall. Gebe in beiden Fällen das Ergebnis in der Konsole aus.

Double Converter: Lösung

```
class NumberConverter {  
    static double toDouble(String number) throws NotANumberException {  
        try {  
            return Double.parseDouble(number);  
        } catch (NumberFormatException e) {  
            throw new NotANumberException(number);  
        }  
    }  
  
    private static void printNumber(String number) { // Hilfsmethode #DRY  
        try {  
            System.out.println(toDouble(number));  
        } catch (NotANumberException e) {  
            System.err.println("Cannot convert number: " + e.getMessage());  
        }  
    }  
  
    static void main(String[] args) {  
        printNumber("a1");  
        printNumber("5.5");  
    }  
}
```

```
class NotANumberException extends Exception {  
    public NotANumberException(String message) {  
        super(message);  
    }  
}
```

Average Exception

Schreibe eine Methode `calculateAverage`. Diese Methode akzeptiert eine `Collection` von Ganzzahlen als Parameter und gibt den Durchschnitt aller enthaltenen Zahlen zurück.

```
static double calculateAverage(Collection<Integer> numbers)
```

Falls der Parameter leer ist, soll die Methode eine `IllegalArgumentException` werfen.

Rufe die Methode jeweils für beide Fälle in einer `main`-Methode auf und gebe das Resultat in der Konsole so aus, dass das Programm nicht abstürzt.

Average Exception: Lösung

```
class AverageCalculator {  
  
    static double calculateAverage(Collection<Integer> numbers) {  
        if (numbers.isEmpty()) {  
            throw new IllegalArgumentException("Collection is empty");  
        }  
  
        int sum = 0;  
        for (int num : numbers) {  
            sum += num;  
        }  
        return (double) sum / numbers.size();  
    }  
  
    static void main(String[] args) {  
        try {  
            System.out.println(calculateAverage(List.of(1, 2, 3)));  
            System.out.println(calculateAverage(List.of()));  
        } catch (IllegalArgumentException e) {  
            System.err.println("Error: " + e.getMessage());  
        }  
    }  
}
```

```
// Alternativ als Stream  
static double calculateAverage(Collection<Integer> numbers) {  
    return numbers.stream()  
        .mapToInt(number -> number)  
        .average()  
        .orElseThrow(() -> new IllegalArgumentException("Collection is empty"));  
}
```